

Extend Java's Concurrent Library with More Concurrent Structures

Design, Analysis and Performance Evaluation of Concurrent Queue Implementations

Aggelos Gkogkosis
University of Crete
Computer Science Department
Professor: Foivos Zakkak

1 Introduction

Concurrent programming is an important part of modern software because many programs need to use more than one thread at the same time. Threads may run in parallel, share data, communicate through memory, and update the same objects. This can improve performance, but it also makes the program harder to reason about. If shared data is not handled correctly, the program may produce wrong results even if each thread looks correct on its own.

One common way to make concurrent programs easier to use is through concurrent data structures. These are data structures that are designed to be accessed by multiple threads without breaking their internal state. Instead of forcing every user of the structure to write synchronization code manually, the synchronization is built into the data structure itself. This is useful because structures such as queues, stacks, maps, and lists are often shared between worker threads.

Queues are especially useful in concurrent programs. They are often used to pass work from producer threads to consumer threads, to store tasks, or to connect different parts of a system. A concurrent queue must preserve the normal FIFO behavior while also allowing multiple threads to insert and remove elements safely. This makes queues a good case study for understanding synchronization, lock-free algorithms, and the tradeoffs between correctness and performance.

Java already provides concurrent queue implementations in the JDK. One important example is `ConcurrentLinkedQueue`, which is a non-blocking FIFO queue based on ideas from the Michael and Scott queue algorithm. Because this queue is part of the standard library, it is a useful baseline for comparison. It gives a real implementation that is already used in Java programs, so custom implementations can be compared against something practical instead of only against a simple theoretical example.

The goal of this project is to implement and study lock-free queue approaches in Java, while also understanding concepts such as `volatile`, `VarHandles`, CAS operations, visibility, atomicity, and the Java Memory Model. The first custom implementation is inspired by the Michael-Scott queue and uses linked nodes, `volatile` references, `VarHandles`, and compare-and-set operations. The second implementation is inspired by the Linked Concurrent Ring Queue idea. It is not a full imple-

mentation of the original LCRQ algorithm, but it uses similar ideas such as linked segments, array-based storage, and atomic operations inside each segment.

Both custom queues were implemented with the same main API as the JDK queue, so they can be tested in the same way. The implementations were added inside JDK 21 and are executed through that version for the main experiments. The standalone source code is also kept in the project so that the queue implementations can be inspected or run without building the modified JDK. This allows the custom queues to be studied directly together with the existing JDK queue implementations.

The project also includes correctness testing and performance evaluation. Correctness is checked by comparing the behavior of the custom queues with the official JDK queue and by testing concurrent behavior. Evaluation is then used to study how the different implementations behave under different thread counts and workloads. The main purpose is to understand the design choices behind these queues and to see how the custom implementations compare with the existing JDK implementation.

2 Background

Concurrency refers to the ability of a system to manage multiple tasks whose executions overlap in time. In multithreaded applications, multiple threads may access and modify shared data concurrently, introducing challenges related to correctness, visibility, atomicity, ordering, and synchronization. As a result, concurrent programming requires mechanisms that ensure predictable behavior when shared state is accessed by multiple execution contexts.

This section introduces the main concepts needed to discuss concurrent programming in Java. Visibility and atomicity are basic concepts that describe how shared state can be observed and updated by multiple threads. Thread safety is discussed in terms of correctness, since a thread-safe class or data structure must preserve its expected behavior under concurrent access. The section also gives a general overview of the Java Memory Model (JMM), synchronization mechanisms such as `volatile`, atomic operations, compare-and-swap (CAS), and `VarHandles`, and the background needed for the later analysis of the JDK concurrent queue implementations.

2.1 Visibility

Visibility describes whether a change made by one thread can be observed by another thread. In a concurrent program, multiple threads may access the same shared variable. However, a value written by one thread is not guaranteed to become immediately visible to other threads unless proper synchronization is used. [1]

This behavior is possible because modern processors, caches, compilers, and the JVM perform various optimizations to improve performance. As a result, different threads may temporarily observe different values for the same variable. [1]

Visibility issues can lead to unexpected behavior. For example, one thread may update a shared flag indicating that a task should stop, while another thread continues to observe the old value and keeps executing. To ensure that updates performed by one thread become visible to others, synchronization mechanisms such

as `volatile` variables, locks, or atomic operations must be used. [1]

```
1 class Task {
2     boolean running = true;
3
4     void stop() {
5         running = false;
6     }
7     void run() {
8         while (running) {
9             // work
10        }
11    }
12 }
13 // Thread A
14 task.run();
15 // Thread B
16 task.stop();
```

Fig. 1. Visibility problem caused by unsynchronized access to a shared variable.

The visibility problem is illustrated in Figure 1. Thread B updates the shared variable `running` by invoking `stop()`, while Thread A continuously checks the same variable inside the `run()` method. Without proper synchronization, the update may not become visible to Thread A, causing the loop to continue executing even though the value has already been changed.

2.2 Atomicity

Atomicity describes whether an operation is executed as a single indivisible action. In concurrent programs, this property is important because multiple threads may attempt to access and modify the same shared data simultaneously. If an operation is not atomic, another thread may observe or modify the data while the operation is still in progress, leading to incorrect results. [1]

Many operations that appear to be a single statement in the source code are actually performed as a sequence of smaller steps. A common example is incrementing a shared counter using the expression `count++`. Although it appears to be a single operation, it is internally executed as three separate actions: reading the current value, incrementing it, and writing the updated value back to memory. This type of operation is commonly referred to as a read-modify-write operation.

When multiple threads perform a read-modify-write operation concurrently, their executions may interleave. As a result, one thread can overwrite the update performed by another thread, causing an increment to be lost. This phenomenon is known as a lost update and represents one of the most common forms of atomicity violation in concurrent programs.

Synchronization mechanisms such as locks, atomic variables, and compare-and-set (CAS) operations are commonly used to ensure that read-modify-write operations are executed atomically and produce correct results under concurrent access.

```
1  int count = 5;
2
3  // Thread A
4  count++;
5
6  // Thread B
7  count++;
```

Fig. 2. Lost update caused by a non-atomic increment operation.

Figure 2 illustrates a lost update caused by concurrent execution of a read-modify-write operation. Both threads may read the same value of `count` before either of them writes the updated result back to memory. For example, if both threads read the value 5, each thread will independently calculate the new value 6. The first thread writes 6 back to memory, but the second thread subsequently writes its own result, which is also 6. As a result, one increment is effectively lost and the final value becomes 6 instead of 7.

2.3 Thread Safety

Thread safety is a fundamental requirement in concurrent systems, yet defining it precisely is not always straightforward. At its core, thread safety is closely related to the concept of correctness. A class is considered correct when it behaves according to its specification and preserves its expected invariants and operational guarantees. [1]

Based on this notion of correctness, a class is considered thread-safe if it continues to behave correctly when accessed concurrently by multiple threads. In other words, the correctness of the class must be preserved regardless of the scheduling or interleaving of thread execution, without requiring additional synchronization by the calling code. Thread-safe classes therefore prevent concurrency-related issues such as race conditions, visibility problems, and data corruption, ensuring predictable and reliable behavior under concurrent access. [1]

2.4 Java Memory Model

The Java Memory Model defines the rules that govern how threads interact through memory. It specifies when writes performed by one thread must become visible to another thread and which ordering guarantees are provided by synchronization actions. These rules are necessary because compilers, processors, and runtime systems may optimize program execution in ways that are not obvious from the source code. [1][2]

The JMM does not require every operation to be immediately visible to every thread. Instead, it defines precise guarantees for correctly synchronized programs. Constructs such as `synchronized`, `volatile`, thread start and termination, and atomic classes create ordering and visibility relationships that allow programmers to reason about shared-memory behavior. [1][2]

2.4.1 Reordering

Reordering refers to the execution of memory operations in an order different from the one in which they appear in the source code. Modern compilers, the JVM, and processors perform such optimizations to improve performance, provided that the behavior of a single thread remains unchanged. [1][2]

In single-threaded programs, reordering is usually invisible to the programmer because the final result remains the same. In concurrent programs, however, multiple threads may observe memory operations in a different order than expected. As a result, one thread may observe the effects of a later operation while still not observing the effects of an earlier one. [1][2]

This behavior can lead to unexpected results when shared variables are accessed without proper synchronization. For this reason, reasoning about the execution order of operations in concurrent programs is often difficult. Synchronization mechanisms restrict certain reorderings and ensure that memory operations are observed in a predictable manner by other threads. [1][2]

```
1  class Reordering {
2      int x = 0, y = 0;
3      public void writer() {
4          x = 1;
5          y = 2;
6      }
7      public void reader() {
8          int r1 = y;
9          int r2 = x;
10     }
11 }
```

Fig. 3. Example illustrating the effects of memory reordering.

Figure 3 demonstrates a simple reordering scenario. A programmer might expect that if a thread observes `y = 2`, it must also observe `x = 1`, since the assignment to `x` appears before the assignment to `y` in the source code. However, due to reordering and visibility effects, another thread may observe the updated value of `y` while still observing the old value of `x`. As a result, `r1` may become 2 while `r2` remains 0.

2.4.2 Happens-before Relationship

The Java Memory Model defines a relationship known as happens-before, which specifies when the actions performed by one thread become visible to another thread. In simple terms, if an action A happens-before an action B, then the thread executing B is guaranteed to observe the effects of A. [1]

The happens-before relationship is the mechanism used by the JMM to provide visibility and ordering guarantees between threads. Without such a relationship,

the JVM, compiler, and processor are free to reorder memory operations, making it difficult to reason about the behavior of concurrent programs. [1]

Several synchronization mechanisms establish happens-before relationships, including locks, `volatile` variables, thread start and termination operations, and atomic variables. These guarantees allow threads to communicate safely through shared memory and ensure that updates performed by one thread become visible to others in a predictable manner. [1]

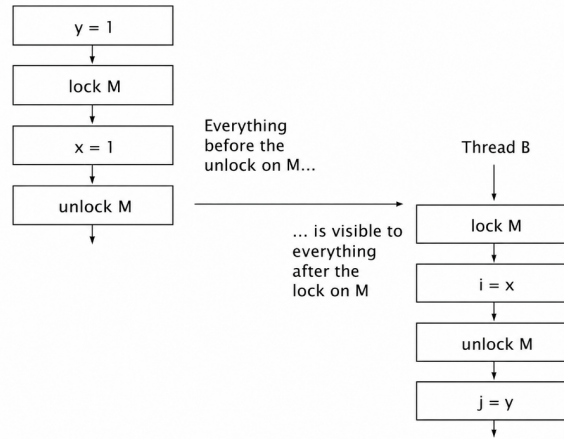


Fig. 4. Happens-before relationship between two synchronized threads.

Figure 4 illustrates a happens-before relationship established through synchronization. Thread A updates the shared variable `x` and subsequently releases lock `M`. When Thread B later acquires the same lock, it is guaranteed to observe all memory updates performed by Thread A before the lock was released. As a result, the value written to `x` becomes visible to Thread B. [1]

2.5 Synchronization Mechanisms

Synchronization mechanisms are the tools used to coordinate access to shared state between threads. In this work, the focus is on mechanisms that are commonly used in lock-free algorithms, because the later sections examine lock-free queue implementations. These mechanisms include `volatile` variables, atomic operations, compare-and-set (CAS) loops, and `VarHandle` operations, which provide the visibility, ordering, and atomicity guarantees needed to update shared references without using locks.

These mechanisms are important for understanding lock-free concurrent queues because they determine how updates to shared nodes, head pointers, tail pointers, and next links become safe under concurrent access. Queue implementations in the JDK rely on these guarantees to avoid data corruption while allowing multiple threads to operate on the same queue without blocking each other through a global lock.

2.5.1 Volatile Variables

The Java language provides the `volatile` keyword as a lightweight synchronization mechanism for shared variables. A variable can be declared as `volatile` by placing the `volatile` modifier before its type. [1]

When a variable is declared as `volatile`, any value written by one thread becomes visible to other threads that subsequently read the same variable. In addition, the JVM is not allowed to reorder memory operations around `volatile` accesses in ways that would violate the visibility guarantees defined by the Java Memory Model. [1]

`volatile` variables are commonly used for status flags, configuration values, or lifecycle events that may be updated by one thread and observed by others. A typical example is a shared flag used to signal that a thread should stop executing. Without the `volatile` modifier, one thread may continue observing an outdated value even after another thread has updated it. [1]

Although `volatile` variables provide visibility guarantees, they do not provide atomicity. Operations such as `count++` are still executed as read-modify-write operations and may suffer from race conditions when accessed concurrently. For this reason, `volatile` variables are generally used when threads only need to observe the most recent value of a variable and no compound update operations are required. [1]

Compared to locks, `volatile` variables are simpler and typically incur lower overhead. However, locks provide both visibility and atomicity guarantees, whereas `volatile` variables only guarantee visibility. [1]

2.5.2 Compare-and-set (CAS)

Compare-And-Set (CAS) is an atomic operation commonly used in lock-free concurrent data structures. The operation receives two values: an expected value and a new value. If the current value stored in memory matches the expected value, it is replaced with the new value. Otherwise, the operation fails and no modification is performed. [3]

The importance of CAS lies in the fact that the comparison and the update are executed as a single atomic action. As a result, no other thread can modify the value between the comparison and the update. This property allows multiple threads to coordinate access to shared data without using locks. [3]

CAS is widely used in modern concurrent algorithms because it provides atomicity while avoiding the overhead associated with locking mechanisms. Many concurrent data structures in the Java platform, including lock-free queues, rely on CAS operations to safely update shared references and state variables. [3]

In Java, CAS operations are exposed through classes such as `AtomicInteger`, `AtomicReference`, and `VarHandle`, which internally rely on hardware-supported atomic instructions provided by the underlying processor architecture. [3]

```

1  AtomicInteger count = new AtomicInteger(5);
2
3  // Thread A
4  boolean successA = count.compareAndSet(5, 6);
5
6  // Thread B
7  boolean successB = count.compareAndSet(5, 6);

```

Fig. 5. Concurrent Compare-And-Set (CAS) operations on a shared variable.

Figure 5 illustrates two threads attempting to update the same shared variable using a CAS operation. Both threads expect the current value of `count` to be 5 and attempt to update it to 6. Only one thread can succeed because the comparison and update are performed atomically. If Thread A updates the value first, the value of `count` becomes 6. When Thread B subsequently executes its CAS operation, the comparison fails because the current value no longer matches the expected value. Instead of overwriting the update performed by Thread A, the operation returns `false`, allowing Thread B to detect the conflict and retry if necessary.

2.5.3 VarHandles

`VarHandles` were introduced in Java 9 as a standard way of performing low-level memory operations on variables. Before their introduction, most lock-free algorithms relied either on atomic classes such as `AtomicInteger` or on the internal `Unsafe` API. `VarHandles` provide similar functionality through an official Java API. [4]

A useful way to think about a `VarHandle` is as a special object that "points" to a specific field of a class. Once created, it can be used to read, write, or atomically update that field. This is particularly useful in concurrent data structures, where multiple threads may need to safely modify the same shared reference. [4]

Unlike atomic classes such as `AtomicInteger` or `AtomicReference`, a `VarHandle` does not store any data itself. Instead, it operates directly on an existing field of an object. In practice, this means that a `VarHandle` can be attached to fields such as `next`, `head`, or `tail` and perform atomic operations on them whenever needed. [4]


```

1 class Node {
2
3     volatile Node next;
4     static final VarHandle NEXT;
5
6     static {
7         try {
8             NEXT = MethodHandles.lookup()
9                 .findVarHandle(Node.class, "next", Node.class);
10        } catch (Exception e) {
11            throw new Error(e);
12        }
13    }
14 }

```

Fig. 6. Concurrent Compare-And-Set (CAS) operations on a shared variable.

```

1 // current.next == null
2
3 //Thread A:
4 NEXT.compareAndSet(current, null, nodeA); // true
5
6 //Thread B:
7 NEXT.compareAndSet(current, null, nodeB); // false

```

Fig. 7. Concurrent Compare-And-Set (CAS) operations on a shared variable.

Figure 6 shows a `VarHandle` associated with the `next` field of a node. Once created, the handle can be used to read, write, or atomically update the field without requiring additional wrapper objects such as `AtomicReference`.

Figure 7 illustrates two threads attempting to update the same reference. Both threads expect the value of `next` to be null. Thread A succeeds first and updates the field to `nodeA`. When Thread B performs the same operation, the expected value no longer matches the current value, causing the CAS operation to fail. Instead of overwriting the existing update, Thread B can retry using the latest state of the data structure.

2.6 Michael-Scott Concurrent Queue

The Michael-Scott Queue is one of the most well-known lock-free queue algorithms and has influenced many modern concurrent queue implementations, such as Java's `ConcurrentLinkedQueue`. The algorithm implements a FIFO queue using a singly-linked list and relies on atomic Compare-And-Swap (CAS) operations instead of locks to coordinate concurrent updates.

The queue maintains two shared references, `Head` and `Tail`, as well as a special dummy node that is created during initialization. The dummy node does not contain user data and is used to simplify the implementation by avoiding special cases when the queue is empty or contains only a single element. [5]



Fig. 8. Initial state of a Michael-Scott Queue.

New elements are inserted at the end of the linked list. To perform an insertion, a thread first creates a new node and then attempts to atomically attach it to the current tail using a CAS operation. If another thread modifies the queue first, the CAS operation fails and the thread retries using the updated state of the queue. Once the new node has been successfully linked, the Tail pointer is advanced to the newly inserted node. [5]

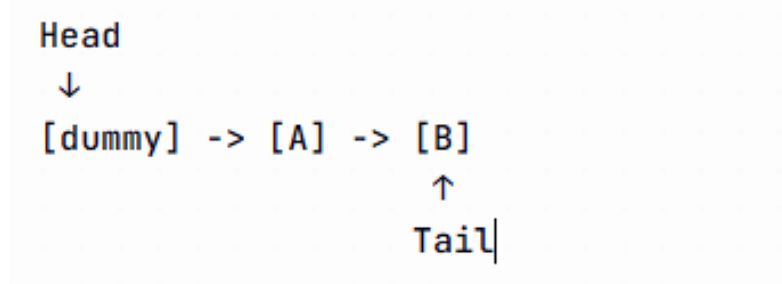


Fig. 9. Michael-Scott Queue after inserting two elements.

Elements are removed from the beginning of the queue. Since Head always points to the dummy node, the first real element is located at head.next. A dequeue operation removes an element by atomically advancing the Head pointer to the next node in the list. The removed node effectively becomes the new dummy node. [5]

One interesting aspect of the algorithm is that threads may help each other complete operations. For example, if a thread inserts a node but is interrupted before updating the Tail pointer, another thread can detect that the pointer is outdated and advance it on behalf of the original thread. This helping mechanism improves overall progress and is one of the key reasons the algorithm remains lock-free. [5]

The algorithm does not use locks, mutexes, or synchronized blocks. Instead, it relies entirely on CAS operations. When a CAS operation fails, it usually means that another thread has successfully completed an update. As a result, even though an individual thread may need to retry, the system as a whole continues making progress. This property makes the Michael-Scott Queue a lock-free data structure and allows it to scale efficiently under concurrent workloads. [5]

2.7 Linked Concurrent Ring Queue

The Linked Concurrent Ring Queue (LCRQ) was proposed as an evolution of the Michael-Scott Queue with the goal of improving scalability under high contention.

While the Michael-Scott Queue relies on a linked list of nodes and CAS operations on shared pointers, the LCRQ combines the Michael-Scott design with concurrent ring buffers and Fetch-And-Add (FAA) operations. The main idea is to reduce contention by spreading concurrent operations across many positions in a ring instead of forcing all threads to compete for the same memory locations.

At a high level, an LCRQ can be viewed as a Michael-Scott Queue where each node of the linked list is replaced by a Concurrent Ring Queue (CRQ). Instead of storing individual elements inside linked-list nodes, each CRQ stores many elements inside a circular array. [6]

Michael-Scott Queue

Node -> Node -> Node

LCRQ

CRQ -> CRQ -> CRQ

Fig. 10. Conceptual relationship between the Michael-Scott Queue and the LCRQ.

This design significantly reduces contention because threads are distributed across many slots of a ring buffer rather than repeatedly updating the same shared pointers. [6]

A key component of the algorithm is the Fetch-And-Add (FAA) operation. Unlike CAS, which may fail when multiple threads attempt to update the same value concurrently, FAA always succeeds and returns a unique position to each thread. [6]

For example, assume the `tail` counter initially contains `tail = 5` and three threads execute `index = FAA(tail, 1)`. The result is that Thread A receives position 5, Thread B receives position 6, Thread C receives position 7, and `tail` becomes 8. [6]

Thread A -> 5

Thread B -> 6

Thread C -> 7

tail becomes 8

Fig. 11. Ticket allocation using Fetch-And-Add.

A useful way to think about FAA is as taking a queue ticket. Every thread receives a unique number and can immediately proceed to its assigned position without competing with the others. [6]

This is one of the main reasons why LCRQ scales better under contention. In a traditional CAS loop, many threads may repeatedly fail and retry while attempting to update the same shared variable. With FAA, every thread obtains a unique position on the first attempt. [6]

Each CRQ consists of a circular array of slots. Each slot stores metadata in addition to the actual value. Most importantly, every slot maintains an index (*idx*) which identifies the logical position associated with that slot. [6]

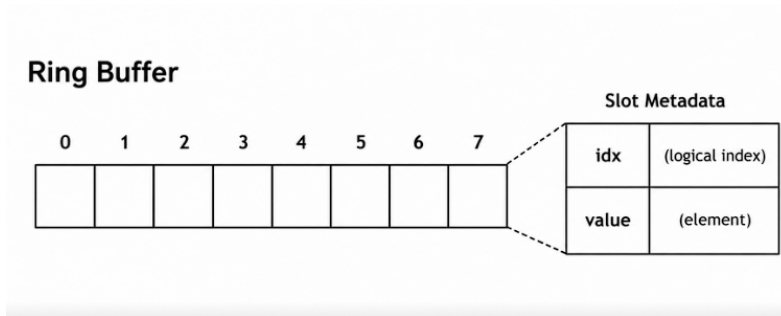


Fig. 12. Ring buffer slots used by a Concurrent Ring Queue.

This is necessary because the ring buffer is reused continuously. For example, if the ring contains eight positions, then indices 2 and 10 both map to the same physical slot, because $2 \% 8 = 2$ and $10 \% 8 = 2$. [6]

Without additional information, the algorithm would not be able to distinguish whether a slot corresponds to an old operation or a new one. The stored index allows threads to determine exactly which logical position a slot currently represents. [6]

To insert an element, a thread first reserves a position using `ticket = FAA(tail, 1)`. It then computes the corresponding slot using `slot = ticket % R`, where *R* is the ring size. [6]

For example, if `tail = 0`, then Thread A receives ticket 0 and slot 0, Thread B receives ticket 1 and slot 1, Thread C receives ticket 2 and slot 2, and Thread D receives ticket 3 and slot 3. [6]

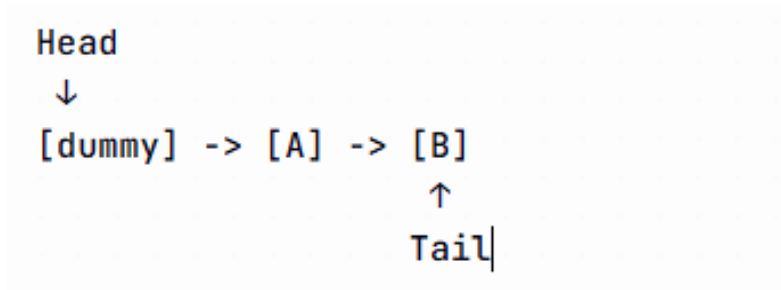


Fig. 13. Threads distributed across different ring positions.

Unlike the Michael-Scott Queue, where multiple threads may attempt to update

the same `tail.next` pointer, the LCRQ spreads concurrent operations across many slots. This dramatically reduces contention because threads are typically working on different positions of the ring. [6]

Once a suitable slot is found, the element is inserted using an atomic update operation. The paper describes this using a double-word compare-and-swap (CAS2), which atomically updates both the slot metadata and the stored value. [6]

Removal follows a similar approach. Instead of advancing through a linked list, a thread reserves a dequeue position using `ticket = FAA(head, 1)`. The ticket determines which slot should be examined. If a valid value is found, the slot is atomically marked as consumed and the value is returned to the caller. [6]

As a result, both enqueue and dequeue operations primarily consist of obtaining a ticket using FAA, mapping the ticket to a ring position, and performing an atomic update on the selected slot. This avoids the repeated retries typically observed in CAS-based linked-list queues under heavy contention. [6]

Eventually a ring may become full or reach a state where further insertions are no longer possible. When this happens, the ring is marked as closed and a new CRQ is allocated. [6]

The new ring is then attached using the same linked-list approach employed by the Michael-Scott Queue. [6]

```
Thread A -> slot0
Thread B -> slot1
Thread C -> slot2
Thread D -> slot3
```

Fig. 14. Linking multiple Concurrent Ring Queues.

This is the reason for the name Linked Concurrent Ring Queue. The algorithm combines two ideas: the linked structure of the Michael-Scott Queue and the high-throughput ring-buffer design of a Concurrent Ring Queue. [6]

In other words, the LCRQ can be viewed as a Michael-Scott Queue whose nodes are themselves concurrent ring buffers. [6]

The main advantage of the LCRQ is that contention is distributed across a large number of slots rather than concentrated on a small number of shared pointers. While the Michael-Scott Queue relies heavily on CAS operations on `Head` and `Tail`, the LCRQ uses FAA to assign unique positions and only performs atomic updates on individual ring slots. This design allows the algorithm to achieve significantly higher throughput under heavily concurrent workloads and is one of the reasons it is often considered a natural evolution of the Michael-Scott Queue on modern multicore systems.

3 Implementation

The main topic of this project is the construction of a Michael-Scott-inspired lock-free queue and an LCRQ-inspired queue approach. The first implementation follows

the basic idea of the Michael-Scott algorithm more directly, using linked nodes and atomic pointer updates. The second implementation is inspired by the Linked Concurrent Ring Queue idea, but it should not be considered a full implementation of the original LCRQ paper. It uses some of the same ideas, such as linked segments, ring-like storage, and atomic updates, in order to study how this style behaves in practice.

The goal is not only to write the queues, but also to compare them. For this reason, the implementations are tested and measured against each other and against the official JDK `ConcurrentLinkedQueue`. This gives a baseline from the Java standard library and makes it easier to see whether the custom approaches behave correctly and how their performance changes with different thread counts.

3.1 Custom `MSConcurrentLinkedQueue` implementation

The first custom queue implementation is the Michael-Scott-inspired queue. In the source code this class is named `MSQueue`. The reason for this implementation is to build a simple lock-free queue with the same general idea used by `ConcurrentLinkedQueue`: a linked list of nodes, a head pointer, a tail pointer, and atomic updates instead of locks. The official `ConcurrentLinkedQueue` is a good comparison point because it is an unbounded, thread-safe, non-blocking FIFO queue from the JDK and its implementation is based on the Michael and Scott queue design. [7]

The queue stores its elements in linked `Node` objects. Each node has a final `item` field and a volatile `next` field. The queue itself also has volatile `head` and `tail` references. The constructor creates one dummy node and makes both `head` and `tail` point to it. This dummy node is useful because the first real value is always stored in `head.next`, so the code does not need a special case for the first insertion.

The implementation uses `VarHandles` for the shared references that must be updated atomically. In the static block, `HEAD`, `TAIL`, and `NEXT` are created with `MethodHandles.lookup()`. These handles are then used for `compareAndSet` operations on `head`, `tail`, and `next`. The volatile fields give visibility between threads, while the CAS operations are used to change shared links only if the expected value is still the current one. If another thread has already changed the queue, the CAS fails and the operation retries.

The most important methods are `offer` and `poll`. This is also where most papers about concurrent queues focus, because these are the enqueue and dequeue operations. They define where items enter and leave the queue, where the main contention happens, and where the main atomic updates are performed. The rest of the API is still implemented, but `offer` and `poll` are the core of the lock-free behavior.

In `offer`, the method first rejects `null` values and creates a new node for the element. Then it repeatedly reads the current `tail` and the `next` field of that tail node. If `next` is `null`, the tail really looks like the last node, so the method tries to link the new node using `NEXT.compareAndSet`. If this CAS succeeds, the element has been inserted into the linked list. After that, the method also tries to move the

tail pointer forward with `TAIL.compareAndSet`. This second CAS is a helping step, so even if it fails, the insertion has already happened.

```

1 public boolean offer(E e) {
2     Objects.requireNonNull(e);
3     Node<E> node = new Node<>(e);
4
5     for (;;) {
6         Node<E> t = tail;
7         Node<E> next = t.next;
8
9         if (t == tail) {
10             if (next == null) {
11                 if (NEXT.compareAndSet(t, null, node)) {
12                     TAIL.compareAndSet(this, t, node);
13                     return true;
14                 }
15             } else {
16                 TAIL.compareAndSet(this, t, next);
17             }
18         }
19         Thread.onSpinWait();
20     }
21 }
22 }

```

Fig. 15. Offer operation in the custom MSQueue implementation.

If `offer` sees that `tail.next` is not null, then another thread has already inserted a node but the `tail` pointer is behind. In that case, the method helps by trying to advance `tail` to `next`. This is an important part of the Michael-Scott style: threads do not only wait for their own operation, they can also help complete the state left by another thread. When a CAS fails, the loop continues and `Thread.onSpinWait()` is used before retrying.

In `poll`, the method repeatedly reads `head`, `tail`, and `head.next`. If `head.next` is null, then there is no real node after the dummy node and the queue is empty, so the method returns null. If `head` and `tail` point to the same node but `next` is not null, then the `tail` pointer is behind and the method helps move it forward.

```

1 public E poll() {
2     for (;;) {
3         Node<E> h = head;
4         Node<E> t = tail;
5         Node<E> next = h.next;
6
7         if (h == head) {
8             if (next == null) {
9                 return null;
10            }
11
12            if (h == t) {
13                TAIL.compareAndSet(this, t, next);
14                continue;
15            }
16
17            E item = next.item;
18
19            if (HEAD.compareAndSet(this, h, next)) {
20                h.next = h;
21                return item;
22            }
23        }
24        Thread.onSpinWait();
25    }
26 }
27 }

```

Fig. 16. Poll operation in the custom MSQueue implementation.

When the queue contains an element, `poll` reads the value from `next.item` and then tries to move `head` from the old dummy node to `next` with a CAS on the

HEAD VarHandle. If this CAS succeeds, the value is returned. The old dummy node is then self-linked with `h.next = h`. In this implementation this is used as a simple cleanup step, similar to the idea of helping garbage collection by making old nodes easier to detach from the active queue path.

The rest of the API was implemented so that the custom queue can be tested against the official `ConcurrentLinkedQueue` with the same calls. The implemented operations include `add`, `addAll`, `contains`, `isEmpty`, `iterator`, `offer`, `peek`, `poll`, `remove`, `size`, `splititerator`, `toArray`, and `toArray(T[] a)`. The supported API follows the methods shown in the JDK documentation. [7]

- `boolean add(E e)`: inserts the specified element at the tail of this queue.
- `boolean addAll(Collection<? extends E> c)`: appends all of the elements in the specified collection to the end of this queue, in the order returned by the collection's iterator.
- `boolean contains(Object o)`: returns `true` if this queue contains the specified element.
- `boolean isEmpty()`: returns `true` if this queue contains no elements.
- `Iterator<E> iterator()`: returns an iterator over the elements in this queue in proper sequence.
- `boolean offer(E e)`: inserts the specified element at the tail of this queue.
- `E peek()`: retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
- `E poll()`: retrieves and removes the head of this queue, or returns `null` if this queue is empty.
- `boolean remove(Object o)`: removes a single instance of the specified element from this queue, if it is present.
- `int size()`: returns the number of elements in this queue.
- `Splititerator<E> splititerator()`: returns a `Splititerator` over the elements in this queue.
- `Object[] toArray()`: returns an array containing all of the elements in this queue, in proper sequence.
- `<T> T[] toArray(T[] a)`: returns an array containing all of the elements in this queue, in proper sequence, using the runtime type of the given array.

Some of these methods are simple wrappers around the main operations. For example, `add` calls `offer`, and `isEmpty` uses `peek`. Other methods traverse the linked nodes from `head.next`, such as `size`, `contains`, and `iterator`. The `toArray` and `splititerator` methods use a snapshot list, because they need a stable collection

of the visible elements at that moment. The `remove(Object o)` method searches for a matching node and then uses CAS on the previous node's next pointer to unlink it.

Having the same API is useful because the same tests can be applied to the official queue, the custom Michael-Scott-inspired queue, and the LCRQ-inspired queue. If all three queues receive the same operations, then their returned values, final contents, and iteration order can be compared directly. Performance measurements are then used to observe how the queues behave with different numbers of threads, especially when many threads call `offer` and `poll` at the same time.

3.2 Custom `LinkedConcurrentRingQueue` implementation

The second custom queue implementation is `LinkedConcurrentRingQueue`. This implementation was found after searching for efficient lock-free linked queue implementations. The idea looked useful for this project because it combines the linked structure of a Michael-Scott-style queue with array-based segments. The implementation here is not a full implementation of the original LCRQ algorithm. It takes some important ideas from it, mainly the linked segments, the use of positions inside an array, and the use of atomic operations to reduce contention. The purpose is to compare this approach with the Michael-Scott-inspired queue and with the official JDK `ConcurrentLinkedQueue`. [6]

The queue is made from linked `Segment` objects. Each segment contains an `Object[]` array called `items`. Instead of allocating one linked node for every single element, a segment can store many elements. The queue keeps two `volatile` references, `headSeg` and `tailSeg`, which point to the current head and tail segments. Each segment also has its own `head`, `tail`, `closed`, and `next` fields. This makes the queue look like a linked list of bounded array blocks.

The original LCRQ uses a real ring buffer and also uses metadata inside each slot. Our implementation is simpler. The segment capacity is rounded to a power of two, but the current code does not reuse the same segment forever like a full circular ring. When a segment reaches its capacity, it is marked as closed and a new segment is linked after it. So it is better to describe this as an LCRQ-inspired segmented queue, not as a complete LCRQ implementation.

The implementation uses `VarHandles` for both segment references and array cells. The handles `HEAD_SEG`, `TAIL_SEG`, and `SEG_NEXT` are used to move between linked segments. The handles `SEG_HEAD`, `SEG_TAIL`, and `SEG_CLOSED` are used for the state inside each segment. The `ARRAY` handle is created with `MethodHandles.arrayElementVarHandle(Object[].class)` and is used to atomically read and update positions inside the array.

A very important part of this implementation is the ticket-style reservation. In the code, this is done with `getAndAdd`, not with a normal read. For example, `enqueue` inside a segment uses `SEG_TAIL.getAndAdd(this, 1L)`. This works like a Fetch-And-Add operation: every thread receives a different number and therefore a different array position. This is different from a CAS loop where many threads may try to update the same shared pointer and fail repeatedly. Here, the atomic increment gives each thread its own slot attempt.

The code also uses `getAcquire` in many places, but for a different reason. `getAcquire` is used when the code only needs to read a shared value, such as the current `tailSeg`, `headSeg`, next segment, or an array cell. It gives the needed acquire ordering for seeing values published by other threads, without doing a full update operation. So the FAA-style part is `getAndAdd`, while `getAcquire` is mostly used for safe observation of shared state.

In `offer`, the outer queue first reads the current `tailSeg` through the `TAIL_SEG` handle. It then calls `offer` on that segment. If the segment accepts the element, the method returns `true`. If the segment is full or closed, the outer queue checks `SEG_NEXT`. If there is no next segment, it creates a new `Segment` and tries to attach it using `SEG_NEXT.compareAndSet`. If another thread already attached a segment, the method helps by moving `tailSeg` forward.

```

1      public E poll() {
2          for (;;) {
3              Segment<E> h = (Segment<E>) HEAD_SEG.getAcquire(this);
4
5              Object r = h.poll();
6
7              if (r != Segment.SEGMENT_EMPTY) {
8                  return (E) r;
9              }
10
11             Segment<E> next = (Segment<E>) SEG_NEXT.getAcquire(h);
12
13             if (next == null) {
14                 return null;
15             }
16
17             HEAD_SEG.compareAndSet(this, h, next);
18             Thread.onSpinWait();
19         }
20     }

```

Fig. 17. Offer operation in the custom `LinkedConcurrentRingQueue` implementation.

Inside a segment, `offer` first checks whether the segment is closed. Then it reserves a position by calling `SEG_TAIL.getAndAdd`. If the returned ticket is greater than or equal to the segment capacity, the segment is closed and the insert fails, so the outer queue can move to another segment. Otherwise, the ticket is used as the array index. The element is inserted with `ARRAY.compareAndSet(items, index, EMPTY, e)`. This means that the thread only writes the element if the slot is still marked as `EMPTY`.

The original LCRQ paper describes a double-word compare-and-swap, usually called CAS2, where the slot metadata and the value can be changed together. Java does not give a simple portable `VarHandle` operation for doing CAS2 on two independent fields at the same time. For that reason, this implementation uses a simpler representation: each array position stores one object state. A slot can contain `EMPTY`, `SKIPPED`, or a real element. This does not fully match the original algorithm, but it keeps the main idea that each slot is updated atomically.

In `poll`, the outer queue reads the current `headSeg` and tries to remove an element from that segment. If the segment returns a real value, `poll` returns it to the caller. If the segment reports `SEGMENT_EMPTY`, the outer queue checks whether

there is a next segment. If there is no next segment, the whole queue is empty at that moment and `poll` returns `null`. If a next segment exists, the method tries to advance `headSeg` with `HEAD_SEG.compareAndSet`.

```

1  public boolean offer(E e) {
2      Objects.requireNonNull(e);
3
4      for (;;) {
5          Segment<E> t = (Segment<E>) TAIL_SEG.getAcquire(this);
6
7          if (t.offer(e)) {
8              return true;
9          }
10
11         Segment<E> next = (Segment<E>) SEG_NEXT.getAcquire(t);
12
13         if (next == null) {
14             Segment<E> n = new Segment<>(segmentCapacity);
15
16             if (SEG_NEXT.compareAndSet(t, null, n)) {
17                 TAIL_SEG.compareAndSet(this, t, n);
18             }
19         } else {
20             TAIL_SEG.compareAndSet(this, t, next);
21         }
22
23         Thread.onSpinWait();
24     }
25 }

```

Fig. 18. Poll operation in the custom `LinkedConcurrentRingQueue` implementation.

Inside a segment, `poll` first reads the segment head and tail. If head is already greater than or equal to tail, then there is no reserved value to read from that segment. Otherwise, the method reserves a dequeue position with `SEG_HEAD.getAndAdd`. This is the same ticket idea as `enqueue`, but applied to removals. The selected slot is then updated with `ARRAY.getAndSet(items, index, SKIPPED)`. If the slot contained a real element, that element is returned. If it contained `EMPTY` or `SKIPPED`, the method knows that the slot did not produce a value and continues or reports that the segment is empty.

The markers `EMPTY` and `SKIPPED` are important for the simplified design. `EMPTY` means that no producer has successfully placed a value in the slot yet. `SKIPPED` means that a consumer reached the slot and marked it as no longer useful. This helps the queue avoid returning the same element twice and also lets producers detect that a slot was already skipped under contention.

The methods `peek`, `contains`, `remove`, `iterator`, `spliterator`, and `toArray` are also implemented, so the custom queue exposes the same API shape as `ConcurrentLinkedQueue`. They are not the main lock-free algorithm, but they are needed for API comparison tests. Most of them work by scanning the linked segments and reading live array slots with `ARRAY.getAcquire`. The `remove` method searches for a matching value and then tries to replace it with `SKIPPED` using `ARRAY.compareAndSet`.

The main reason for implementing this version is to see whether spreading operations across array positions can reduce contention compared with a pure linked-node queue. In the Michael-Scott-style implementation, many threads often interact with the same `tail`, `head`, or `next` pointers. In this implementation, the linked segment pointers still exist, but most `enqueue` and `dequeue` operations first try to work

inside the current segment. This is the part that makes it interesting for comparison, even though it is still a simplified LCRQ-inspired approach.

4 Evaluation

Before looking at throughput or scalability, the first thing I wanted to check was whether the two custom queues are actually valid and whether they behave the same way as the JDK `ConcurrentLinkedQueue`. For that reason, I first ran a set of custom correctness tests for both `MSConcurrentLinkedQueue` and `LinkedConcurrentRingQueue`. These tests compare the custom queues directly with `ConcurrentLinkedQueue` and check the basic queue API step by step. In practice this means checking operations such as `add`, `offer`, `poll`, `peek`, `contains`, `remove`, `iterator`, `spliterator`, and `toArray`, along with null handling and general FIFO behavior. I also used randomized differential tests with many mixed operations, so that both custom queues and the official JDK queue receive the same sequence of calls and can be compared on returned values and final contents.

I also wanted this check to be done in a setup that is as close as possible to real library use, and not only as standalone classes in my own source folder. Because of that, the two implementations were installed directly into a local OpenJDK 21 build under `java.util.concurrent` as `MSConcurrentLinkedQueue` and `LinkedConcurrentRingQueue`. The JDK build used for these tests reports itself as `OpenJDK 21.0.12-internal`. On top of the sequential and randomized API checks, I also ran multithreaded stress tests across different thread counts and small `JCStress` tests for simple race patterns such as two concurrent `offer` calls and `offer/poll` interleavings. The point of all this was to make sure that before talking about performance, the two custom queues already look correct and behave like the official JDK queue in the cases tested.

This section should define how the queue implementations will be evaluated. The evaluation should include throughput, latency, scalability across thread counts, and behavior under different producer-consumer ratios.

4.1 Experimental Setup

The experiments were run on my laptop, a Lenovo 20VG system with an AMD Ryzen 5 4500U with Radeon Graphics. The machine has 8 GB of DDR4 system memory, 384 KB of L1 cache, 3 MB of L2 cache, and 8 MB of L3 cache. The queue implementations were tested inside the local OpenJDK 21.0.12-internal build where the two custom queues were installed under `java.util.concurrent`.

During the experiments, the laptop was connected to power, so it was not running on battery. I also kept turbo boost disabled in order to reduce frequency changes during the measurements. This was checked with `sudo turbostat` before the runs.

4.2 Metrics

For the performance part, I used both some simple custom timing tests and JMH. The custom timing code is under `test/benchmarks/custom`. These tests are useful because they are easy to read and they show clearly what is being measured. In particular, I measured sequential `offer+poll` runs with 1,000,000 operations, and I also measured threaded `offer/poll` runs with 1, 4, 8, 16, and 32 threads. In the threaded tests, the code also checks things like checksum, null polls, and remaining elements, so the timing is not separated from basic correctness.

I also used JMH, the Java Microbenchmark Harness. JMH is the standard Java microbenchmark framework and it helps avoid some common benchmarking mistakes by doing warm-up, controlled measurement iterations, and repeated execution under the JVM. Here it was used mainly to measure the round-trip queue cost, where each benchmark operation does an `offer()` followed by a `poll()`, and to compare the same three implementations in a more controlled setup.

The three queues compared in these measurements are the official JDK `ConcurrentLinkedQueue`, my custom `MSConcurrentLinkedQueue`, and `LinkedConcurrentRingQueue`. For the final comparison, I report execution times in milliseconds (ms), so lower values mean better performance.

Threads	Custom LCRQ (ms)	Custom MSQueue (ms)	Official Queue (ms)	Fastest
Sequential (1,000,000 offer+poll pairs)	31.904	47.756	42.228	Custom LCRQ
1 thread (100,000 ops/thread)	20.132	5.921	7.849	Custom MSQueue
4 threads (100,000 ops/thread)	58.200	117.061	79.441	Custom LCRQ
8 threads (100,000 ops/thread)	66.880	279.731	175.364	Custom LCRQ
16 threads (100,000 ops/thread)	140.373	688.225	364.683	Custom LCRQ
32 threads (100,000 ops/thread)	319.679	1346.625	872.045	Custom LCRQ

Fig. 19. Initial timing results in milliseconds for the three queue implementations.

5 Conclusion

From the first results, the LCRQ-inspired queue looks like the fastest option, especially once contention starts going up. A reasonable explanation is that it spreads the work across segment slots instead of forcing as many threads to fight on the same linked pointers, so it likely suffers from fewer failed CAS retries than the other linked-node style queues.

The custom `MSConcurrentLinkedQueue` is generally slower than the official JDK `ConcurrentLinkedQueue`, which is not very surprising. The official implementation has more low-level tuning in `offer()` and `poll()` and in general it is a much more optimized production implementation. At the same time, the custom

MS queue can look better in the single-thread case, probably because the algorithm is simpler and has less overhead when there is no contention.

So the overall picture is fairly clear: the LCRQ-style segmented approach looks stronger under concurrent load, while the official JDK queue is still the better tuned version of the Michael-Scott style design. The custom MS implementation is still useful because it shows the core idea more directly and makes the behavior of this kind of lock-free queue easier to study.

References

- [1] Brian Goetz et al. *Java Concurrency in Practice*. Addison-Wesley, 2006.
- [2] Jeremy Manson and Brian Goetz. *JSR 133 (Java Memory Model) FAQ*. February 2004.
- [3] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. 2008.
- [4] OpenJDK. *JEP 193: Variable Handles*.
- [5] Maged M. Michael and Michael L. Scott. *Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms*. 1996.
- [6] Adam Morrison and Yehuda Afek. *Fast Concurrent Queues for x86 Processors*. PPOPP, 2013.
- [7] Oracle. *ConcurrentLinkedQueue (Java Platform SE 8)*. <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ConcurrentLinkedQueue.html>.